

BÀI 08: CÁC LỚP CƠ SỞ VÀ CÁC LỚP CẤU TRÚC DỮ LIỆU

Mục tiêu

Về kiến thức:

- Giúp sinh viên hiểu và thiết kế các giao diện, cài đặt các lớp giao diện và triển khai chúng cho bài toán cụ thể. Hiểu được mảng các lớp cơ sở; các lớp cấu trúc dữ liệu

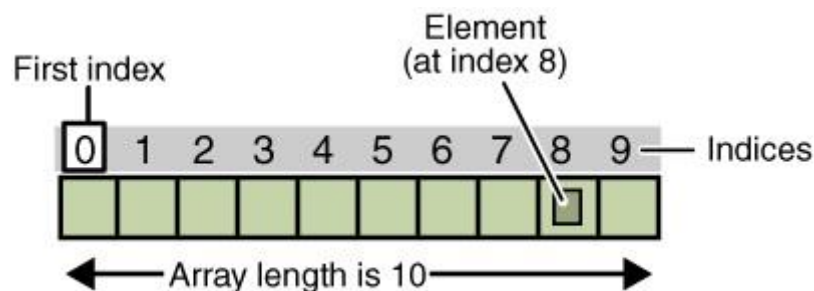
Về kỹ năng:

Giúp sinh viên vận dụng mảng các lớp cơ sở; các lớp cấu trúc để viết chương trình giải quyết bài toán cụ thể.

8.1. Cấu trúc mảng

8.1.1. Khai báo mảng

Mảng (array) là tập hợp các phần tử có cùng kiểu dữ liệu và có số phần tử là cố định (không thay đổi được số phần tử sau khi tạo lập). Trong java mảng là kiểu tham chiếu do vậy ta phải khởi tạo mảng trước khi sử dụng.



Hình 8.1. Mảng

Khai báo

<Kiểu các phần tử> [] <Tên mảng>;

<Kiểu các phần tử> <Tên mảng> [];

Trong đó <Kiểu các phần tử> có thể là kiểu nguyên thủy hoặc là kiểu lớp (tên lớp).

Kích thước của mảng chưa xác định khi khai báo

8.1.2. Tạo lập mảng

Để tạo lập đối tượng mảng thì phải xác định số phần tử của mảng đó.

<Tên mảng> = new <Kiểu các phần tử> [<Số phần tử>];

Trong đó:

<Kiểu các phần tử> là kiểu tương thích với kiểu mà mảng đã khai báo.
Giá trị cực tiểu của <Số phần tử> là 0.

Kết hợp cả khai báo với tạo lập mảng như sau:

<Kiểu các phần tử 1> <Tên mảng>[] = new <Kiểu các phần tử 2>[<Số phần tử>];

Khai báo và tạo lập mảng <Tên mảng> kiểu <Kiểu các phần tử 1> để chứa <Số phần tử> các phần tử có <Kiểu các phần tử 2>.

<Kiểu các phần tử 1> và <Kiểu các phần tử 2> là hai kiểu tương thích với nhau. Nếu mảng đó có kiểu nguyên thủy thì hai kiểu đó phải trùng nhau. Nếu là kiểu lớp thì <Kiểu các phần tử 2> là lớp con của <Kiểu các phần tử 1> (một lớp cũng được xem là lớp con của chính nó).

Khi một mảng được tạo lập thì tất cả các phần tử của nó được khởi tạo giá trị mặc định (0 hoặc 0.0 đối với kiểu số, false đối với kiểu boolean, null cho kiểu lớp).

Ví dụ:

float mangFloat[] = new float[20]; // Các phần tử được gán trị mặc định

Object[] dayDen = new BongDen[5]; // Các phần tử được gán mặc định null khởi tạo các mảng.

Khai báo, tạo lập và gán ngay các giá trị ban đầu:

<Kiểu các phần tử >[] <Tên mảng> = {<Các giá trị ban đầu>};

Ví dụ: *int[] mangInt = {1, 3, 5, 7, 9};*

Tạo ra *mangInt* có 5 phần tử với phần tử đầu có giá trị là 1 (*mangInt*[0] = 1), phần tử thứ hai là 3 (*mangInt*[1] = 3) , v.v.

Object[] dayDT = {new BongDen(), new BongDen(), null};

char[] charArr = {"a", "b", "a"}; // mảng 3 ký tự và hoàn toàn khác với "aba"

8.1.3. Truy cập các phần tử của mảng

Truy cập một phần tử mảng bằng cách sử dụng số chỉ số (index) trong đó $0 \leq \text{index} < (\text{số phần tử mảng})$. Phần tử thứ i của mảng là phần tử có chỉ số i - 1 (<Tên mảng>[i - 1]). Nếu truy cập vượt quá phạm vi xác định của mảng sẽ gặp ngoại lệ *ArrayIndexOutOfBoundsException*

Truy cập vào phần tử của mảng: <tên mảng>[i]

Ví dụ:

public class TruyCapArray1 {

```

public static void main(String[] args) {
    String[] cars = { "Honda", "BMW", "Ford", "Mazda" };
    System.out.println(cars[0]);
}
}

```

Kết quả:

Honda

Để thay đổi giá trị của một phần tử cụ thể trong mảng, ta cũng dùng chỉ số:

```

public class TruyCapArray2 {
    public static void main(String[] args) {
        String[] cars = { "Honda", "BMW", "Ford", "Mazda" };
        // thay đổi phần tử đầu tiên của mảng cars
        cars[0] = "Morning";
        // hiển thị phần tử đầu tiên của mảng cars
        System.out.println("Phần tử đầu tiên: " + cars[0]);
    }
}

```

Kết quả:

Phần tử đầu tiên: Morning

Để biết có bao nhiêu phần tử trong một mảng, ta sử dụng thuộc tính **length**:

```

public class DodaiArray1 {
    public static void main(String[] args) {
        String[] cars = { "Honda", "BMW", "Ford", "Mazda" };
        System.out.println("Độ dài của mảng cars là: " + cars.length);
    }
}

```

Kết quả:

4

8.1.4. Duyệt các phần tử của mảng trong Java

Có 2 cách để duyệt các phần tử của mảng:

- Sử dụng vòng lặp for
- Sử dụng foreach

Sử dụng vòng lặp for

Ta có thể lặp qua các phần tử mảng bằng vòng lặp for và sử dụng thuộc tính length để chỉ định số lần vòng lặp sẽ chạy.

Ví dụ:

```
public class DuyệtArray1 {  
    public static void main(String[] args) {  
        String[] cars = { "Honda", "BMW", "Ford", "Mazda" };  
        for (int i = 0; i < cars.length; i++) {  
            System.out.println(cars[i]);  
        }  
    }  
}
```

Kết quả:

Honda

BMW

Ford

Mazda

Sử dụng foreach

```
public class DuyệtArray2 {  
    public static void main(String[] args) {  
        String[] cars = { "Honda", "BMW", "Ford", "Mazda" };  
        for (String car : cars) {  
            System.out.println(car);  
        }  
    }  
}
```

Kết quả:

Honda

BMW

Ford

Mazda

So sánh for với foreach

Vòng lặp for là một cấu trúc điều khiển để chỉ định phép lặp cho phép lệnh được thực thi nhiều lần.	Vòng lặp foreach là một cấu trúc điều khiển để duyệt các mục trong một mảng hoặc một tập hợp.
Vòng lặp for có thể được sử dụng để truy xuất một tập hợp các phần tử cụ thể.	Vòng lặp foreach không thể được sử dụng để truy xuất một tập hợp các phần tử cụ thể.
Vòng lặp for khó đọc và ghi hơn vòng lặp foreach.	Vòng lặp foreach dễ đọc hơn vòng lặp for.
Vòng lặp for được sử dụng như một vòng lặp mục đích chung.	Vòng lặp foreach được sử dụng cho mảng và tập hợp.

8.1.5. Mảng nhiều chiều

Định nghĩa:

<Kiểu các phần tử> [][]...[] <Tên mảng>;

hoặc

<Kiểu các phần tử> <Tên mảng> [][]...[];

Ví dụ: `int[][] mang1; // Mảng hai chiều`

tương đương với `int mang1[][];`

tương đương với `int[] mang1[];`

Khai báo với thiết lập mảng nhiều chiều tương tự mảng đơn.

`int[][] mangA = new int[4][5]; // Ma trận có 4 hàng, 5 cột`

Truy nhập tới từng phần tử : `mangA[i][j]`.

Kích thước của `mangA` là `mangA.length = 4` và mỗi phần tử của nó lại là mảng có kích thước là `mangA[i].length = 5`, $i = 1, 2, \dots, 4$.

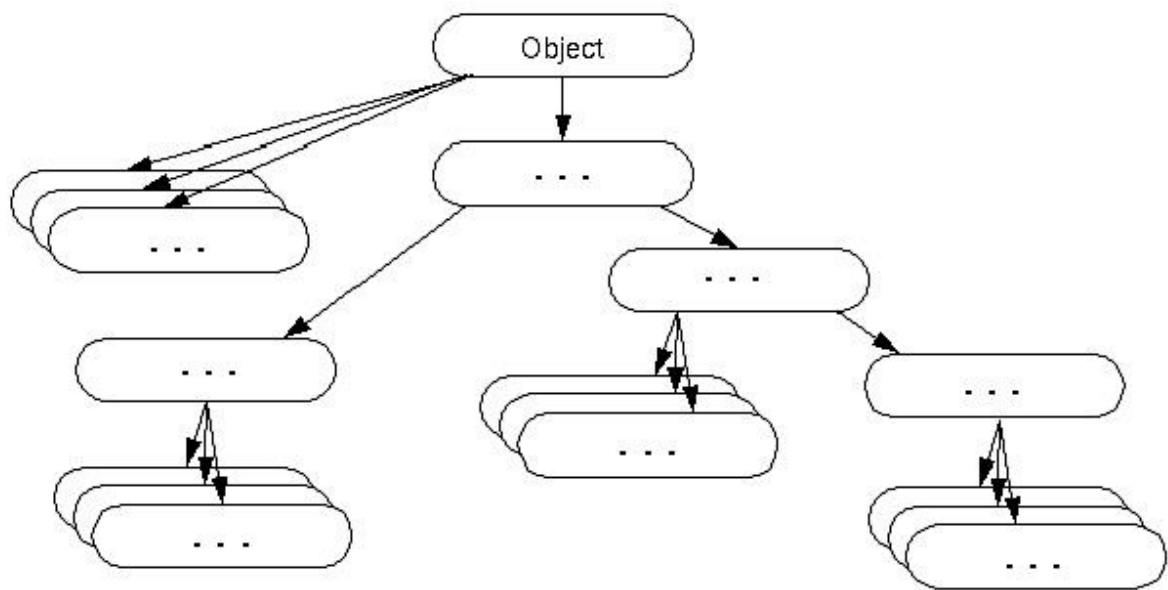
```
double[][] maTran = {  
    {1, 2, 3, 4}, // hàng 1  
    {0, 2, 0, 0}, // hàng 2  
    {0, 0, 3, 0}, // hàng 3  
    {0, 0, 0, 4}, // hàng 4  
};
```

Lưu ý: Các mảng trong mảng nhiều chiều không nhất thiết phải có số phần tử giống nhau.

8.2. Các lớp cơ sở trong gói java.lang

8.2.1. Lớp Object

Tất cả các lớp được xây dựng trong các chương trình Java đều hoặc là trực tiếp hoặc gián tiếp được mở rộng từ lớp Object. Đây là lớp cơ sở nhất, định nghĩa hầu như tất cả những phương thức phân cơ bản để các lớp con cháu của nó sử dụng trực tiếp hoặc viết đè. Object cung cấp các phương thức sau: *int hashCode()*



Hình 5.2 Lớp object

Khi các đối tượng được lưu vào các bảng băm (hash table), hàm này có thể sử dụng để xác định duy nhất giá trị cho mỗi đối tượng. Điều này đảm bảo tính nhất quán của hệ thống khi thực hiện chương trình.

Sử dụng lớp Object là hữu ích nếu ta muốn tham chiếu bất kỳ đối tượng nào mà ta chưa biết kiểu dữ liệu của đối tượng đó. Chú ý rằng biến tham chiếu của lớp cha có thể tham chiếu đến đối tượng của lớp con được gọi là *upcasting*.

Ví dụ: giả sử phương thức getObject() trả về một đối tượng nhưng nó có thể là bất kỳ kiểu nào như Employee, Student, ... chúng ta có thể sử dụng biến tham chiếu của lớp Object để tham chiếu tới đối tượng đó.

Object obj=getObject(); // chúng ta không biết đối tượng gì được trả về từ phương thức này.

Lớp Object cung cấp các phương thức như trong bảng sau:

Phương thức	Mô tả
<code>public final Class getClass()</code>	trả về đối tượng lớp Class của đối tượng hiện tại. Từ lớp Class đó có thể lấy được các thông tin metadata của class hiện tại.
<code>public int hashCode()</code>	trả về số hashcode cho đối tượng hiện tại.
<code>public boolean equals(Object obj)</code>	so sánh đối tượng đã cho với đối tượng hiện tại.
<code>protected Object clone() throws CloneNotSupportedException</code>	tạo và trả về bản sao chép (clone) của đối tượng hiện tại.
<code>public String toString()</code>	trả về chuỗi ký tự đại diện của đối tượng hiện tại.
<code>public final void notify()</code>	đánh thức một luồng, đợi trình giám sát của đối tượng hiện tại.
<code>public final void notifyAll()</code>	đánh thức tất cả các luồng. đợi trình giám sát của đối tượng hiện tại.
<code>public final void wait(long timeout) throws InterruptedException</code>	làm cho Thread hiện tại đợi trong khoảng thời gian là số mili giây cụ thể, tới khi Thread khác thông báo (gọi phương thức <code>notify()</code> hoặc <code>notifyAll()</code>).
<code>public final void wait(long timeout, int nanos) throws InterruptedException</code>	làm cho Thread hiện tại đợi trong khoảng thời gian là số mili giây và nano giây cụ thể, tới khi Thread khác thông báo (gọi phương thức <code>notify()</code> hoặc <code>notifyAll()</code>).
<code>public final void wait() throws InterruptedException</code>	làm Thread hiện tại đợi, tới khi Thread khác thông báo (gọi phương thức <code>notify()</code> hoặc <code>notifyAll()</code>).
<code>protected void finalize() throws Throwable</code>	Được gọi bởi Garbage Collector trước khi đối tượng bị dọn rác.

8.2.2. Các lớp nguyên thủy

Các giá trị nguyên thủy không phải là đối tượng trong Java. Để có thể thao tác được trên các giá trị nguyên thủy (giá trị số, kí tự và logic) thì gói java.lang cung cấp các lớp bao gói (Wrapper) cho từng kiểu dữ liệu nguyên thủy (gọi tắt là lớp bao). Các lớp bao có những đặc tính chung sau:

*** Các toán tử tạo lập chung:** Các lớp bao (trừ lớp Character chỉ có một toán tử tạo lập) đều có hai toán tử tạo lập:

- Toán tử tạo lập sử dụng giá trị nguyên thủy để tạo ra đối tượng tương ứng.

Character charObj = new Character('a');

Boolean boolObj = new Boolean(true);

Integer intObj = new Integer(2002);

Float floatObj = new Float(3.14F);

Double doubleObj = new Double(3.14);

- Toán tử thứ hai: chuyển các đối tượng lớp String biểu diễn cho các giá trị nguyên thủy về các lớp tương ứng. Các toán tử này sẽ ném ra ngoại lệ `NumberFormatException` khi giá trị String truyền vào Toán tử tạo lập không hợp lệ.

Boolean boolObj = new Boolean("true");

Integer intObj = new Integer("2002");

Float floatObj = new Float("3.14F");

Double doubleObj = new Double("3.14");

- Các hàm tiện ích chung: *valueOf(String s)*, *toString()*,

typeValue(), *equals()*

*** Mỗi lớp (trừ lớp Character)** đều định nghĩa hàm static *valueOf(String s)* trả lại đối tượng tương ứng. Các hàm này ném ra ngoại lệ `NumberFormatException` khi giá trị String truyền vào phương thức không hợp lệ.

Boolean boolObj = Boolean.valueOf("true");

Integer intObj = Integer.valueOf("2002");

Float floatObj = Float.valueOf("3.14F");

Double doubleObj = Double.valueOf("3.14");

*** Các lớp viết đè hàm toString()** trả lại là các đối tượng String biểu diễn cho các giá trị nguyên thủy ở dạng xâu.

```
String charStr String boolStr String intStr String doubleStr =  
doubleObj.toString;
```

*** Các lớp viết đè hàm equals()** để thực hiện so sánh bằng nhau của các đối tượng nguyên thủy.

```
Character charObj = new Character(„a“);  
boolean charTest = charObj.equals(„b“); // false  
Integer intObj1 = Integer.valueOf(“2010”);  
boolean intTest = intObj.equals(intObj1); // false
```

Lớp Boolean

Lớp này định nghĩa hai đối tượng Boolean.TRUE, Boolean.FALSE biểu diễn cho hai giá trị nguyên thủy true và false tương ứng.

Lớp Character

Lớp Character định nghĩa hai giá trị cực tiểu, cực đại Character.MIN_VALUE, Character.MAX_VALUE và các giá trị kiểu ký tự Unicode. Ngoài ra lớp này còn định nghĩa một số hàm static để xử lý trên các ký tự:

```
static boolean isLowerCase(char ch)// true nếu ch là ký tự thường  
static boolean isUpperCase(char ch)// true nếu ch là ký tự viết hoa  
static boolean isDigit(char ch) // true nếu ch là chữ số  
static boolean isLetter(char ch)// true nếu ch là chữ cái  
static boolean isLetterOrDigit(char ch) // true nếu ch là chữ hoặc là số  
static char toUpperCase(char ch)// Chuyển ch về chữ viết hoa  
static char toLowerCase(char ch)// Chuyển ch về chữ viết thường  
static char toTitleCase(char ch)// Chuyển ch về dạng tiêu đề.
```

Các lớp bao kiểu số

Các lớp Byte, Short, Integer, Long, Float, Double là các lớp con của lớp Number. Trong các lớp này đều xác định hai giá trị:

<Lớp bao>.MIN_VALUE

<Lớp bao>.MAX_VALUE là các giới hạn của các số trong kiểu đó.

Ví dụ: byte minByte = Byte.MIN_VALUE;

 int maxInt = Integer.MAX_VALUE;

Trong mỗi lớp bao có hàm `typeValue()` để chuyển các giá trị của các đối tượng nguyên thủy về giá trị số:

```
// -128
// 2147483647
byte byteValue()
short shortValue()
int intValue()
long longValue()
float floatValue()
double doubleValue()
```

Trong mỗi lớp bao còn có hàm `static parseType(String s)` để chuyển các giá trị được biểu diễn dưới dạng xâu về các giá trị số:

```
byte value1 = Byte.parseByte("16");
int value2 = Integer.parseInt("2002");
double value3 = Double.parseDouble("3.14");
```

Ví dụ: Viết chương trình để nhập vào một dãy số tùy ý và sắp xếp theo thứ tự tăng dần.

```
import java.io.*;
class SapXep {
    static int[] day;
    static void nhap() {
        String str;
        int n = day.length;
        DataInputStream stream = new DataInputStream(System.in);
        System.out.println("Nhập vào " + n + " số nguyên");
        for (int k = 0; k < n; k++) {
            try {
                System.out.print(k + ": ");
                str = stream.readLine();
                day[k] = Integer.valueOf(str).intValue();
            } catch (IOException e) {
                System.err.println("I/O Error!");
            }
        }
    }
}
```

```

        }
    }
}

static void hienThi() {
    int n = day.length;
    for (int k = 0; k < n; k++) {
        System.out.print(day[k] + " ");
    }
    System.out.println();
}

static void sapXep() {
    int x, max, k;
    for (int i = day.length - 1; i > 0; i--) {
        max = day[i];
        k = i;
        for (int j = 0; j < i; j++) {
            if (max < day[j]) {
                max = day[j];
                k = j;
            }
        }
        day[k] = day[i];
        day[i] = max;
    }
}

public static void main(String[] args) {
    String str;
    int n;
    DataInputStream stream = new DataInputStream(System.in);
    System.out.print("\nCho biet bao nhieu so nhap vao: ");
    try {
        str = stream.readLine();
    }
}

```

```

    } catch (IOException e) {
        System.err.println("I/O Error!");
        str = "0";
    }
    n = Integer.valueOf(str).intValue();
    SapXep.day = new int[n];
    nhap();
    sapXep();
    System.out.println("Day so duoc sap xep: ");
    hienThi();
}
}

```

8.2.3. Lớp Math

Lớp Math nằm trong gói java.lang (gói mặc định) do vậy không cần phải thêm câu lệnh import ở đầu chương trình để có thể sử dụng lớp này. Các hàm này được viết là các phương thức tĩnh nên ta không cần phải tạo ra thể hiện của lớp Math. Bảng sau liệt kê một số phương thức tĩnh trong lớp Math:

Tên phương thức	Mô tả	Kiểu tham số	Kiểu trả về
sin(arg)	tính sin của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double
cos(arg)	tính cos của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double
tan(arg)	tính tang của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double
asin(arg)	tính $\sin^{-1}$ (arcsin) của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double trong hệ radians
acos(arg)	tính $\cos^{-1}$ (arccosin) của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double trong hệ radians

atan(arg)	tính $\tan^{-1}$ (arctang) của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double trong hệ radians
atan2 (arg1,arg2)	tính $\tan^{-1}$ (arctang) của arg1/arg2	arg1,arg2 là các biểu thức kiểu double thể hiện một cung theo radians	double trong hệ radians
abs(arg)	tính trị tuyệt đối của arg	arg là một biểu thức kiểu int, long, float, hoặc double	The same type as the argument
max (arg1,arg2)	Nhận về giá trị lớn trong hai tham số	arg1, arg2 là một biểu thức kiểu int, long, float, hoặc double	Nhận về kiểu cùng Kiểu với tham số
min (arg1,arg2)	Nhận về giá trị nhỏ trong hai tham số	arg1, arg2 là một biểu thức kiểu int, long, float, double	Nhận về kiểu cùng kiểu với tham số
ceil(arg)	Nhận về giá trị nguyên nhỏ hơn hoặc bằng arg	arg là biểu thức kiểu float hoặc double	double
floor(arg)	Nhận về giá trị nguyên lớn hơn hoặc bằng arg	arg là biểu thức kiểu float hoặc double	double
round(arg)	Trả về giá trị nguyên gần arg nhất, giá trị này chính là giá trị của arg sau khi đã làm tròn	arg là biểu thức kiểu float hoặc double	Nhận về kiểu int nếu arg kiểu float, nhận về kiểu long nếu arg kiểu double
rint(arg)	Giống round(arg)	arg là biểu thức kiểu double	double
sqrt(arg)	tính căn bậc hai của arg	arg là biểu thức kiểu double	double
pow (arg1,arg2)	tính $\arg_1^{\arg_2}$	Cả arg1 và arg2 là các biểu thức kiểu double	double
exp(arg)	tính earg	arg là biểu thức kiểu double	double

log(arg)	tính logarithm cơ số e của arg	arg là biểu thức kiểu double	double
random()	Nhận về một số giả ngẫu nhiên nằm trong khoảng [0, 1)	Không có tham số	double

Bảng 5.1 Một số phương thức tính trong lớp Math

8.2.4. Lớp String

Lớp String dựng cho những chuỗi ký tự bất biến (chuỗi chỉ đọc), khi được khởi tạo giá trị đầu, giá trị đó không thay đổi được nữa.

Phương thức khởi tạo (Constructor):

String(String s)

String(<Kieu>[] mang)

String(StringBuffer buf)

Ví dụ:

```
byte[] bytes = {97, 98, 98, 97};
char[] characters = {„a“, „b“, „b“, „a“};
StringBuffer buff = new StringBuffer("abba");
String byteStr = new String(bytes); // Chuyển mảng bytes về chuỗi
String charStr = new String(characters); // charStr = "abba"
String buffStr = new String(buff);
String str = new String("abba");
```

Có thể sử dụng: String str = "Dung dong den toi";

Ví dụ: Khởi tạo chuỗi trong Java

```
public class StringInitialization {
    public static void main(String a[]) {
        String objStr_01 = "This is a string object";
        String objStr_02 = new String("This is also string object");
        char[] arrChar = {'V', 'N', 'L', 'T', 'V', 'E', 'S'};
        String objStr_03 = new String(arrChar);
        String objStr_04 = objStr_03 + " This is another String object";

        System.out.println("objStr_01: " + objStr_01);
        System.out.println("objStr_02: " + objStr_02);
        System.out.println("objStr_03: " + objStr_03);
        System.out.println("objStr_04: " + objStr_04);
    }
}
```

Kết quả chạy chương trình:

objStr_01: This is a string object

objStr_02: This is also string object

objStr_03: VNLIVES

objStr_04: VNLIVES This is another String object

Đọc từng ký tự

+ *int length()*: kích thước chuỗi.

+ *char charAt(int index)*: cho lại ký tự thứ index (bắt đầu từ 0) của chuỗi hiện thời ($0 \leq \text{index} < \text{length}()$). (ngoại lệ *StringIndexOutOfBoundsException*).

So sánh các chuỗi

boolean equals(Object obj): kiểm tra xem đối tượng lớp String và đối tượng ở tham số có cùng tập ký hiệu hay không

boolean equalsIgnoreCase(String str2) kiểm tra xem đối tượng lớp String và đối tượng ở tham số có cùng tập ký hiệu hay không (phân biệt chữ hoa và chữ thường)

boolean startsWith(String str)

boolean endsWith(String str)

Hai hàm này cho kết quả true nếu chuỗi bắt đầu (kết thúc) bằng đối số str.

int compareTo(String str2)

int compareTo(Object obj)

+ 0 nếu chuỗi hiện thời bằng chuỗi đối số,

+ nhỏ hơn 0, nếu chuỗi đó nhỏ hơn chuỗi đối số theo thứ tự từ điển,

+ lớn hơn 0, nếu chuỗi đó lớn hơn chuỗi đối số theo thứ tự từ điển,

Hàm *compareTo()* thứ hai thực hiện tương tự như hàm đầu nếu đối số là chuyển về được đối tượng chuỗi, ngược lại sẽ cho qua để xử lý ngoại lệ *ClassCastException*.

Chuyển đổi chuỗi:

String toUpperCase()

String toLowerCase()

Ghép chuỗi

String concat(String str)

Tìm các ký tự và các chuỗi con Nếu không tìm được các hàm này sẽ cho kết quả là -1:

int indexOf(int ch) Tìm chỉ số của lần xuất hiện đầu tiên của *ch* trong xâu.

int indexOf(int ch, int fromIndex) Tìm chỉ số của lần xuất hiện đầu tiên của *ch* trong xâu bắt đầu từ *fromIndex*.

int indexOf(String str) Tìm chỉ số của lần xuất hiện đầu tiên của xâu con *str* trong xâu hiện thời.

int indexOf(String str, int fromIndex) Tìm chỉ số của lần xuất hiện đầu tiên của xâu con *str* bắt đầu từ *fromIndex* trong xâu hiện thời.

int lastIndexOf(int ch) Tìm chỉ số của lần xuất hiện cuối cùng của ký tự *ch* trong xâu.

int lastIndexOf(int ch, int fromIndex) Tìm chỉ số của lần xuất hiện cuối cùng của *ch* bắt đầu từ *fromIndex* trong xâu.

int lastIndexOf(String str) Tìm chỉ số của lần xuất hiện cuối cùng của xâu con *str* trong xâu hiện thời.

int lastIndexOf(String str, int fromIndex) Tìm chỉ số của lần xuất hiện cuối cùng của xâu con *str* bắt đầu từ *fromIndex* trong xâu hiện thời.

String replace(char cu, char moi) Thay tất cả các lần xuất hiện của ký tự *cu* bằng ký tự *moi* trong xâu.

Trích ra các xâu con

String trim() tạo một xâu mới, các ký tự “trắng” (những ký tự có giá trị nhỏ hơn giá trị dấu cách „ „) ở trong xâu đều bị loại bỏ.

String substring(int startIndex) Cho kết quả là một xâu con được triết ra từ vị trí *startIndex* đến cuối của xâu.

String substring(int startIndex, int endIndex) Cho kết quả là một xâu con được triết ra từ vị trí *startIndex* đến vị trí *endIndex* của xâu. (ngoại lệ *StringIndexOutOfBoundsException*.)

Chuyển các đối tượng của Object về String

static String valueOf(Object obj)

static String valueOf(char[] characters)

Hai hàm này được nạp chồng để chuyển các đối tượng *obj* và mảng các ký tự *characters* về xâu ký tự.

static String valueOf(boolean b)

static String valueOf(char c)

Hàm được nạp chồng để chuyển hai giá trị boolean: true, false về xâu ký tự “true”, “false” tương ứng.

static String valueOf(int i)

static String valueOf(long l)

static String valueOf(float f)

static String valueOf(double d)

Các hàm này được nạp chồng để chuyển các giá trị số về dạng xâu ký tự.

Ví dụ: Sử dụng các hàm cơ bản của chuỗi.

```
package javaandroidvn;
public class JavaAndroidVn {
    public static void main(String[] args) {
        String str = "Android.Vn Android.Vn";
        System.out.println("str = " + str);
        // Lấy từ vị trí số 8 tới cuối cùng của chuỗi
        System.out.println("str.substring(8) = " + str.substring(8));
        // Lấy từ vị trí số 3 tới vị trí số 9
        System.out.println("str.substring(3,9) = " + str.substring(3, 9));
        // Độ dài chuỗi:
        System.out.println("Độ dài chuỗi: str.length() = " + str.length());
        // Lấy ra ký tự trong chuỗi theo chỉ số
        char ch;
        ch = str.charAt(4);
        System.out.println("str.charAt(4) = " + ch);
        // Thay 1 ký tự bằng ký tự khác trong chuỗi:
        System.out.println("Thay tất cả ký tự 'n' bằng ký tự 'x' = " +
str.replace('n', 'x'));
        // Tìm chuỗi "And" là chuỗi con của chuỗi str, thay kết quả đầu tiên bằng
chuỗi "xxx"
        System.out.println("Thay And đầu tiên bằng xxx = " +
str.replaceFirst("And", "xxx"));
        // Thay toàn bộ chuỗi "And" của chuỗi str bằng chuỗi "xxx"
        System.out.println("Thay tất cả And bằng xxx = " +
str.replaceAll("And", "xxx"));
        // Chuyển thành chữ thường:
        System.out.println("str chuyển về viết thường: " + str.toLowerCase());
        // Chuyển thành chữ hoa:
        System.out.println("str chuyển về viết hoa: " + str.toUpperCase());
        // Loại bỏ khoảng trống 2 bên chuỗi
        String str1 = " " + str + " ";
        System.out.println(" Android.Vn Android.Vn --> " + str1.trim());
    }
}
```

```

package javaandroidvn;
public class JavaAndroidVn {
    public static void main(String[] args) {
        String str = "Android.Vn Android.Vn";
        System.out.println("str = " + str);
        // Lấy từ vị trí số 8 tới cuối cùng của chuỗi
        System.out.println("str.substring(8) = " + str.substring(8));
        //Lấy từ vị trí số 3 tới vị trí số 9
        System.out.println("str.substring(3,9) = " + str.substring(3, 9));
        //Độ dài chuỗi:
        System.out.println("Độ dài chuỗi: str.length() = " + str.length());
        //Lấy ra ký tự trong chuỗi theo chỉ số
        char ch;
        ch = str.charAt(4);
        System.out.println("str.charAt(4) = " + ch);
        // Thay 1 ký tự bằng ký tự khác trong chuỗi:
        System.out.println("Thay tất cả ký tự 'n' bằng ký tự 'x' = " +
str.replace('n', 'x'));
        //Tìm chuỗi "And" là chuỗi con của chuỗi str, thay kết quả đầu
tiên bằng chuỗi "xxx"
        System.out.println("Thay And đầu tiên bằng xxx = " +
str.replaceFirst("And", "xxx"));
        //Thay toàn bộ chuỗi "And" của chuỗi str bằng chuỗi "xxx":
        System.out.println("Thay tất cả And bằng xxx = " +
str.replaceAll("And", "xxx"));
        //Chuyển thành chữ thường:
        System.out.println("str chuyển về viết thường: " +
str.toLowerCase());
        //Chuyển thành chữ hoa:
        System.out.println("str chuyển về viết hoa: " +
str.toUpperCase());
        //Loại bỏ khoảng trống 2 bên chuỗi
        String str1 = " " + str + " ";
        System.out.println(" Android.Vn Android.Vn --> " + str1.trim());
    }
}

package javaandroidvn;
public class JavaAndroidVn {
    public static void main(String[] args) {
        String str = "Android.Vn Android.Vn";
        System.out.println("str = " + str);
        // Lấy từ vị trí số 8 tới cuối cùng của chuỗi
        System.out.println("str.substring(8) = " + str.substring(8));
        //Lấy từ vị trí số 3 tới vị trí số 9
        System.out.println("str.substring(3,9) = " + str.substring(3, 9));
        //Độ dài chuỗi:
        System.out.println("Độ dài chuỗi: str.length() = " + str.length());
        //Lấy ra ký tự trong chuỗi theo chỉ số

```

```

        char ch;
        ch = str.charAt(4);
        System.out.println("str.charAt(4) = " + ch);
        // Thay 1 ký tự bằng ký tự khác trong chuỗi:
        System.out.println("Thay tất cả ký tự 'n' bằng ký tự 'x' = " +
str.replace('n', 'x'));
        //Tìm chuỗi "And" là chuỗi con của chuỗi str, thay kết quả đầu
tiên bằng chuỗi "xxx"
        System.out.println("Thay And đầu tiên bằng xxx = " +
str.replaceFirst("And", "xxx"));
        //Thay toàn bộ chuỗi "And" của chuỗi str bằng chuỗi "xxx":
        System.out.println("Thay tất cả And bằng xxx = " +
str.replaceAll("And", "xxx"));
        //Chuyển thành chữ thường:
        System.out.println("str chuyển về viết thường: " +
str.toLowerCase());
        //Chuyển thành chữ hoa:
        System.out.println("str chuyển về viết hoa: " +
str.toUpperCase());
        //Loại bỏ khoảng trống 2 bên chuỗi
        String str1 = " " + str + " ";
        System.out.println(" Android.Vn Android.Vn --> " + str1.trim());
    }
}

```

8.2.5. Lớp StringBuffer

Lớp StringBuffer được sử dụng đối với những chuỗi ký tự động, có thể thay đổi nội dung.

Tạo lập đối tượng StringBuffer

StringBuffer(String str) Tạo ra đối tượng mới nội dung giống như chuỗi đối số str kích thước bằng kích thước của chuỗi str + 16.

StringBuffer(int length) Tạo ra đối tượng mới chưa có nội dung và đặt kích thước bằng đối số length, nếu đối số lớn hơn 0.

StringBuffer() Tạo ra đối tượng mới chưa có nội dung và đặt kích thước của chuỗi buffer là 16.

Đọc và thay đổi các ký tự trong StringBuffer

int length() Cho số ký tự trong chuỗi buffer.

char charAt(int index)

void setCharAt(int index, char ch)

Hàm đầu đọc một ký tự và hàm thứ hai thay đổi ký tự ở vị trí index trong xâu buffer thành *ch*. Hệ thống sẽ cho qua ngoại lệ *StringIndexOutOfBoundsException* nếu các chỉ số khụng tương thích.

Chuyển StringBuffer sang String

Lớp *StringBuffer* nạp chồng hàm *toString()* để chuyển xâu buffer về xâu của lớp cố định *String*.

Hàm *append()* bổ sung các ký tự vào cuối của xâu.

StringBuffer append(Object obj)

Đối tượng *obj* được chuyển về xâu (bằng hàm *String.valueOf()*) và sau đó bổ sung vào xâu buffer.

StringBuffer append(String str)

StringBuffer append(char[] str)

StringBuffer append(char[] str, int offset, int len)

StringBuffer append(char c)

Các mảng ký tự *str* và ký tự *c* được chuyển về xâu và sau đó bổ sung vào xâu *buffer*

StringBuffer append(boolean b)

StringBuffer append(int i)

StringBuffer append(long l)

StringBuffer append(float f)

StringBuffer append(double d)

Các giá trị nguyên thủy được chuyển về xâu và sau đó bổ sung vào xâu *buffer*.

StringBuffer insert(int offset, Object obj)

StringBuffer insert(int offset, String str)

StringBuffer insert(int offset, char[] str)

StringBuffer insert(int offset, char c)

StringBuffer insert(int offset, boolean b)

StringBuffer insert(int offset, int i)

StringBuffer insert(int offset, long l)

StringBuffer insert(int offset, float f)

StringBuffer insert(int offset, double d)

chèn đối số thứ hai (sau khi được chuyển về dạng xâu bằng cách sử dụng hàm `String.valueOf()`) vào từ vị trí `offset` trong xâu `buffer`.

`StringBuffer delete(int index)` `StringBuffer delete(int start, int end)`

Hàm đầu xóa đi ký tự ở vị trí `index`, hàm sau xóa đi một xâu con kể từ vị trí `start` đến `end`.

`StringBuffer reverse()` đảo ngược xâu

Kiểm soát kích thước của `StringBuffer`

`int capacity()`: Cho lại kích thước (khả năng chứa) của xâu `buffer`.

`void ensureCapacity(int minCap)`: Đảm bảo đủ chỗ cho ít nhất `minCap` ký tự của xâu `buffer`.

`void setLength(int newLen)`: Đặt độ dài của xâu `buffer` là `newLen`.

8.3. Một số cấu trúc dữ liệu phổ dụng

8.3.1. Enumeration

Interface `Enumeration` bản thân nó không phải là cấu trúc dữ liệu, nhưng rất quan trọng bên trong ngữ cảnh sử dụng các cấu trúc dữ liệu khác. Interface `Enumeration` định nghĩa để nhận các thành phần kế tiếp từ cấu trúc dữ liệu.

Các phương thức được khai báo bởi `Enumeration`:

`boolean hasMoreElements()` Khi được triển khai, nó phải trả về `true` trong khi vẫn còn nhiều phần tử để extract, và `false` khi tất cả phần tử đã được liệt kê

`Object nextElement()` Trả về đối tượng kế tiếp trong `Enumeration` như là một tham chiếu generic `Object`

Ví dụ:

```
import java.util.Vector;
import java.util.Enumeration;
public class EnumerationTester {
    public static void main(String args[]) {
        Enumeration days;
        Vector dayNames = new Vector();
        dayNames.add("Sunday");
        dayNames.add("Monday");
        dayNames.add("Tuesday");
        dayNames.add("Wednesday");
        dayNames.add("Thursday");
        dayNames.add("Friday");
        dayNames.add("Saturday");
        days = dayNames.elements();
```

```

        while (days.hasMoreElements()){
            System.out.println(days.nextElement());
        }
    }
}

```

Kết quả:

```

Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday

```

8.3.2. BitSet

Lớp BitSet trong Java triển khai một nhóm các bit hoặc flag mà có thể được thiết lập và xóa một cách riêng rẽ.

Class này rất hữu dụng trong trường hợp bạn muốn lưu trữ một tập các giá trị Boolean và chỉ muốn gán từng bit các giá trị và thiết lập hoặc xóa nó thích hợp.

BitSet định nghĩa hai constructor:

BitSet(): tạo một đối tượng mặc định.

BitSet(int size): cho phép bạn xác định kích cỡ ban đầu của nó, ví dụ như số bit mà nó có thể giữ. Tất cả các bit được khởi tạo về 0.

BitSet triển khai *Cloneable Interface* và định nghĩa các phương thức được liệt kê sau:

void and(BitSet bitSet) AND nội dung của đối tượng BitSet đang triệu hồi với nội dung đã được xác định bởi bitSet. Kết quả được đặt trong đối tượng đang triệu hồi

void andNot(BitSet bitSet): Với mỗi bit trong bitSet, bit tương ứng trong BitSet đang triệu hồi bị xóa.

int cardinality(): Trả về số bit được thiết lập trong đối tượng đang triệu hồi

void clear(): Thiết lập tất cả bit về 0

void clear(int index): Thiết lập tất cả bit được xác định bởi *index* về 0

void clear(int startIndex, int endIndex): Thiết lập tất cả bit được xác định từ *startIndex* tới *endIndex - 1* về 0

Object clone(): Sao chép đối tượng BitSet đang triệu hồi

boolean equals(Object bitSet): Trả về *true* nếu BitSet đang triệu hồi là tương đương với thiết lập bit trong bitSet. Nếu không là *false*

void flip(int index): Đảo ngược các bit được xác định bởi index

void flip(int startIndex, int endIndex): Đảo ngược các bit được xác định từ *startIndex* tới *endIndex - 1*

BitSet get(int startIndex, int endIndex): Trả về một BitSet mà chứa các bit từ *startIndex* tới *endIndex-1*. Đối tượng đang triệu hồi không bị thay đổi.

int length(): Trả về số bit cần thiết để giữ nội dung của BitSet đang gọi. Giá trị này được xác định bởi vị trí của bit cuối cùng.

void set(int index): Thiết lập bit được xác định bởi index.

int size(): Trả về số bit trong đối tượng BitSet đang gọi

void xor(BitSet bitSet): XOR nội dung của đối tượng BitSet đang gọi với nội dung được xác định bởi bitSet. Kết quả được đặt trong đối tượng đang gọi.

Ví dụ:

```
import java.util.BitSet;
public class BitSetDemo {
    public static void main(String args[]) {
        BitSet bits1 = new BitSet(16);
        BitSet bits2 = new BitSet(16);
        // thiết lập một số bit
        for (int i = 0; i < 16; i++) {
            if ((i % 2) == 0) {
                bits1.set(i);
            }
            if ((i % 5) != 0) {
                bits2.set(i);
            }
        }
        System.out.println("Pattern ban đầu trong bits1: ");
        System.out.println(bits1);
        System.out.println("\nPattern ban đầu trong bits2: ");
        System.out.println(bits2);
        // AND bits
        bits2.and(bits1);
        System.out.println("\nbits2 AND bits1: ");
        System.out.println(bits2);
        // OR bits
        bits2.or(bits1);
        System.out.println("\nbits2 OR bits1: ");
        System.out.println(bits2);
    }
}
```

```

        // XOR bits
        bits2.xor(bits1);
        System.out.println("\nbits2 XOR bits1: ");
        System.out.println(bits2);
    }
}

```

Kết quả:

Pattern ban đầu trong bits1:

{0, 2, 4, 6, 8, 10, 12, 14}

Pattern ban đầu trong bits2:

{1, 2, 3, 4, 6, 7, 8, 9, 11, 12, 13, 14}

bits2 AND bits1:

{2, 4, 6, 8, 12, 14}

8.3.3. Vector

Lớp Vector trong Java là tương tự như các mảng dữ liệu Java truyền thống, ngoại trừ việc có thể tăng lưu trữ cho các thành phần mới. Giống như mảng, các thành phần trong đối tượng Vector có thể truy cập bởi index.

Lớp Vector trong Java hỗ trợ 4 constructor:

Vector(): Form đầu tiên tạo một vector mặc định, mà có kích cỡ khởi tạo là 10

Vector(int size): Form thứ hai tạo một vector mà dung lượng khởi tạo được xác định bởi size.

Vector(int size, int incr): Form thứ ba tạo một vector mà dung lượng khởi tạo được xác định bởi size và lượng gia của nó được xác định bởi incr. Lượng gia xác định số phần tử để cấp phát mỗi khi một vector được resize.

Vector(Collection c): Form thứ tư tạo một vector mà chứa các phần tử của collection c.

Vector định nghĩa các phương thức sau:

STT	Phương thức và Miêu tả
1	void add(int index, Object element) Chèn <i>element</i> đã xác định tại vị trí đã cho trong Vector này
2	boolean add(Object o)

	Phụ thêm phần tử đã cho vào cuối của Vector này
3	boolean addAll(Collection c) Thêm tất cả phần tử trong Collection đã xác định vào cuối của Vector này, để mà chúng được trả về bởi Iterator của Collection đã cho đó
4	boolean addAll(int index, Collection c) Chèn tất cả phần tử trong Collection đã xác định vào trong Vector này tại vị trí đã cho
5	void addElement(Object obj) Thêm phần tử đã cho tới cuối của Vector này, tăng kích cỡ nó thêm 1
6	int capacity() Trả về dung lượng hiện tại của Vector này
7	void clear() Gỡ bỏ tất cả phần tử từ Vector này
8	Object clone() Trả về một mô phỏng của Vector này
9	boolean contains(Object elem) Kiểm tra nếu đối tượng đã cho là một phần tử trong Vector này
10	boolean containsAll(Collection c) Trả về true nếu Vector này chứa tất cả phần tử trong Collection đã cho
11	void copyInto(Object[] anArray) Sao chép các thành phần của Vector này vào trong mảng đã cho
12	Object elementAt(int index) Trả về phần tử tại index đã cho

13	Enumeration elements() Trả về một bản liệt kê các phần tử của Vector này
14	void ensureCapacity(int minCapacity) Tăng dung lượng của Vector này, nếu cần thiết, để bảo đảm rằng nó có thể giữ ít nhất số các phần tử được xác định bởi tham số minCapacity
15	boolean equals(Object o) So sánh Object đã cho với Vector này về sự cân bằng
16	Object firstElement() Trả về phần tử đầu tiên (tại chỉ mục 0) của Vector này
17	Object get(int index) Trả về phần tử tại vị trí đã cho trong Vector này
18	int hashCode() Trả về giá trị hash code cho Vector này
19	int indexOf(Object elem) Tìm kiếm sự xuất hiện đầu tiên của tham số đã cho, kiểm tra tính cân bằng bởi sử dụng phương thức equals
20	int indexOf(Object elem, int index) Tìm kiếm sự xuất hiện đầu tiên của tham số đã cho, bắt đầu tìm kiếm tại index, kiểm tra tính cân bằng bởi sử dụng phương thức equals
21	void insertElementAt(Object obj, int index) Chèn đối tượng đã cho như là một phần tử vào Vector này tại index đã cho
22	boolean isEmpty() Kiểm tra nếu Vector này không có phần tử
23	Object lastElement()

	Trả về phần tử cuối cùng của Vector này
24	int lastIndexOf(Object elem) Trả về chỉ mục của sự xuất hiện cuối cùng của đối tượng đã cho trong Vector này
25	int lastIndexOf(Object elem, int index) Tìm kiếm ngược về sau cho đối tượng đã cho, bắt đầu từ index đã xác định, và trả về một chỉ mục
26	Object remove(int index) Gỡ bỏ phần tử tại vị trí đã cho trong Vector này
27	boolean remove(Object o) Gỡ bỏ sự xuất hiện đầu tiên của phần tử đã cho trong Vector này. Nếu Vector này không chứa phần tử đó, nó không bị thay đổi
28	boolean removeAll(Collection c) Gỡ bỏ tất cả phần tử, mà chứa trong Collection đã cho, từ Vector này
29	void removeAllElements() Gỡ bỏ tất cả phần tử từ Vector này và thiết lập kích cỡ về 0
30	boolean removeElement(Object obj) Gỡ bỏ sự xuất hiện đầu tiên (chỉ mục thấp nhất) của tham số từ Vector này
31	void removeElementAt(int index) removeElementAt(int index)
32	protected void removeRange(int fromIndex, int toIndex) Gỡ bỏ từ danh sách này tất cả phần tử mà có index từ fromIndex tới toIndex
33	boolean retainAll(Collection c)

	Chỉ giữ lại phần tử, mà ở trong Collection đã cho, trong Vector này
34	Object set(int index, Object element) Thay thế phần tử tại vị trí đã cho trong Vector này với phần tử đã xác định
35	void setElementAt(Object obj, int index) Thiết lập phần tử tại index đã cho của Vector này thành đối tượng đã xác định
36	void setSize(int newSize) Thiết lập kích cỡ của Vector này
37	int size() Trả về số phần tử trong Vector này
38	List subList(int fromIndex, int toIndex) Trả về một danh sách phụ từ fromIndex tới toIndex
39	Object[] toArray() Trả về một mảng chứa tất cả phần tử trong Vector này theo đúng thứ tự
40	Object[] toArray(Object[] a) Trả về một mảng chứa tất cả phần tử trong Vector này theo đúng thứ tự; kiểu runtime của mảng trả về là mảng đã xác định
41	String toString() Trả về một biểu diễn chuỗi của Vector này, chứa biểu diễn chuỗi của mỗi phần tử
42	void trimToSize() Trim dung lượng của Vector này về kích cỡ hiện tại của vector

Ví dụ:

```
import java.util.*;
```

```

public class VectorDemo {
    public static void main(String args[]) {
        // capacity ban dau la 3, incr la 2
        Vector v = new Vector(3, 2);
        System.out.println("Size ban dau: " + v.size());
        System.out.println("Capacity ban dau: " + v.capacity());
        v.addElement(new Integer(1));
        v.addElement(new Integer(2));
        v.addElement(new Integer(3));
        v.addElement(new Integer(4));
        System.out.println("Capacity sau 4 lan cong la: " + v.capacity());
        v.addElement(new Double(5.45));
        System.out.println("Capacity hien tai: " + v.capacity());
        v.addElement(new Double(6.08));
        v.addElement(new Integer(7));
        System.out.println("Capacity hien tai: " + v.capacity());
        v.addElement(new Float(9.4));
        v.addElement(new Integer(10));
        System.out.println("Capacity hien tai: " + v.capacity());
        v.addElement(new Integer(11));
        v.addElement(new Integer(12));
        System.out.println("Phan tu dau tien: " + (Integer) v.firstElement());
        System.out.println("Phan tu cuoi cung: " + (Integer)
v.lastElement());
        if (v.contains(new Integer(3))) {
            System.out.println("Vector chua 3.");
        }
        // tinh toan so phan tu trong vector.
        Enumeration vEnum = v.elements();
        System.out.println("\nCac phan tu trong Vector:");
        while (vEnum.hasMoreElements()) {
            System.out.print(vEnum.nextElement() + " ");

```

```

    }
    System.out.println();
}
}

```

Kết quả:

Size ban dau: 0
Capacity ban dau: 3
Capacity sau 4 lan cong la: 5
Capacity hien tai: 5
Capacity hien tai: 7
Capacity hien tai: 9
Phan tu dau tien: 1
Phan tu cuoi cung: 12
Vector chua 3.
Cac phan tu trong Vector:
1 2 3 4 5.45 6.08 7 9.4 10 11 12

8.3.4. Stack

Lớp Stack trong Java triển khai một last-in-first-out (LIFO) stack các phần tử.

Stack chỉ định nghĩa constructor mặc định, là tạo một stack trống. Lớp Stack bao gồm tất cả phương thức được định nghĩa bởi lớp Vector, và một số phương thức khác của riêng nó.

Lớp Stack định nghĩa các phương thức sau:

STT	Phương thức và Miêu tả
1	boolean empty() Kiểm tra nếu Stack này là trống. Trả về true nếu nó trống và false nếu stack chứa các phần tử
2	Object peek() Trả về phần tử trên cùng của Stack, nhưng không gỡ bỏ nó
3	Object pop() Trả về phần tử trên cùng của Stack, gỡ bỏ nó

4	Object push(Object element) Đẩy phần tử lên trên cùng của Stack. Cũng trả về phần tử đó
5	int search(Object element) Tìm kiếm phần tử trong Stack. Nếu tìm thấy, offset của nó từ trên cùng của Stack được trả về. Nếu không, nó trả về -1

Ví dụ:

```
import java.util.*;

public class StackDemo {

    static void showpush(Stack st, int a) {
        st.push(new Integer(a));
        System.out.println("push(" + a + ")");
        System.out.println("stack: " + st);
    }

    static void showpop(Stack st) {
        System.out.print("pop -> ");
        Integer a = (Integer) st.pop();
        System.out.println(a);
        System.out.println("stack: " + st);
    }

    public static void main(String args[]) {
        Stack st = new Stack();
        System.out.println("stack: " + st);
        showpush(st, 42);
        showpush(st, 66);
        showpush(st, 99);
        showpop(st);
        showpop(st);
        showpop(st);
        try {
            showpop(st);
        } catch (EmptyStackException e) {
```

```

        System.out.println("empty stack");
    }
}

```

Kết quả:

```

stack: [ ]
push(42)
stack: [42]
push(66)
stack: [42, 66]
push(99)
stack: [42, 66, 99]
pop -> 99
stack: [42, 66]
pop -> 66
stack: [42]
pop -> 42
stack: [ ]

```

pop -> empty stack

8.4. Bài tập mẫu

Bài 1: Khởi tạo một chuỗi trong java

```

public class StringInitialization {
    public static void main(String a[]){
        String objStr_01 = "This is a string object";
        String objStr_02 = new String("This is also string object");
        char[] arrChar = {'V','N','L','T','V','E','S'};
        String objStr_03 = new String(arrChar);
        String objStr_04 = objStr_03 + " This is another String object";
        System.out.println("objStr_01: " + objStr_01);
        System.out.println("objStr_02: " + objStr_02);
        System.out.println("objStr_03: " + objStr_03);
        System.out.println("objStr_04: " + objStr_04);
    }
}

```

Bài 2: So sánh chuỗi trong java

```

package javaandroidvn;

```



```

public class JavaAndroidVn {
    public static void main(String[] args) {
        String str1 = "Android.Vn Android.Vn";
        String str2 = "android.vn android.vn";
        System.out.println("So sánh phân biệt chữ hoa chữ thường:
"+str1.equals(str2));
        System.out.println("So sánh không phân biệt hoa thường:
"+str1.equalsIgnoreCase(str2));
        // So sánh thứ tự a, b, c ....
        String str3 = "abc";
        String str4 = "bcde";
        System.out.println("So sánh 2 chuỗi thường!");
        System.out.println("'+str3.compareTo(str4)); //str3 < str4 , so
sánh trả về -1
        System.out.println("'+str4.compareTo(str3)); // str4 > str3, so
sánh trả về 1
        System.out.println("'+str3.compareTo(str3)); // str3 = str3, so
sánh trả về 0
        System.out.println("So sánh 2 chuỗi hoa và thường");
        String str5 = "ANDroid.Vn";
        String str6 = "android.vn";
        System.out.println("'+str5.compareToIgnoreCase(str6)); //Không
phân biệt hoa và thường
        System.out.println("'+str5.compareTo(str6)); // Phân biệt chữ hoa
và chữ thường!
        System.out.println("Chuỗi này có là tập con của chuỗi kia
không?");
        String str7 = "android.vn";
        String str8 = "vn";
        //Quá trình so sánh có phân biệt chữ hoa và chữ thường!
        System.out.println("Chuỗi vn nằm ở vị trí thứ
"+str7.indexOf(str8)+" của chuỗi android.vn");
        //Khi so sánh không tìm thấy thì sẽ trả về -1
        str8 = "TN";
        System.out.println("Chuỗi TN nằm ở vị trí thứ
"+str7.indexOf(str8)+" của chuỗi android.vn");
        System.out.println("Chuỗi này có bắt đầu hay kết thúc bằng chuỗi
kia không?");
        str7 = "android.vn";
        String str9 = "and";
        String str10 = "roid.vn";
        System.out.println("str7.startsWith(str9) =
"+str7.startsWith(str9));
        System.out.println("str7.endsWith(str10) =
"+str7.endsWith(str10));
    }
}

```

8.5. Bài tập thực hành

Bài 1: Để xử lý các văn bản, người ta xây dựng lớp VanBan có thuộc tính riêng là một xâu ký tự. Hãy:

1. Xây dựng lớp VanBan có:

- +Các toán tử tạo lập : VanBan(), VanBan(String st).
- +Phương thức đếm số từ của xâu thuộc tính trong lớp hiện tại.
- +Phương thức đếm số từ kết thúc bởi ký tự ‘G’ (không phân biệt chữ hoa/thường).

2. Xây dựng các phương thức của lớp VanBan để chuẩn hoá xâu thuộc tính trong lớp hiện tại theo các tiêu chí sau:

- + Ở đầu và cuối của xâu không có ký tự trống
- + Ở giữa xâu, tại vị trí bất kỳ không tồn tại hai ký tự trống đứng liền nhau.

Bài 2: Xây dựng lớp Xau có thuộc tính riêng là : String st và các phương thức sau :

- public void CharacterDuplicate():Tìm và in ra các ký tự xuất hiện nhiều hơn một lần trong String cho trước không phân biệt chữ hoa hay chữ thường. Nếu các ký tự trong xâu đều là duy nhất thì xuất ra “NO”. Ví dụ chuỗi “Java” thì có ký tự ‘a’ hoặc String “JaVA” cũng có kết quả tương tự.

Gợi ý: Sử dụng HashMap

- boolean StringAnagram(String st1) (st1 là xâu đối số): Kiểm tra 2 xâu hiện thời st và xâu st1 có là đảo ngược của nhau hay không. Nếu có trả về giá trị là true ngược lại trả về giá trị là false. Ví dụ “word” và “drow” là 2 chuỗi đảo ngược nhau.

- boolean CharacterUnique(char ch): Tìm ký tự chỉ xuất hiện một lần trong chuỗi, nếu ký tự ch chỉ xuất hiện một lần duy nhất trong xâu st thì trả về true, trong các trường hợp còn lại trả về false

- String ReverseString(): Trả về một xâu đảo ngược của xâu hiện thời (st)
boolean Stringnotcontainsdigit() Kiểm tra một chuỗi có chứa chữ số hay không, nếu có in ra false ngược lại true.

Ví dụ

“abc”, “” => true

“1abc”, “abc1”, “123”, “a1bc”, null => false

- Gợi ý: Sử dụng java regex, đây cũng là cách ngắn gọn nhất. Hoặc có thể dùng vòng lặp và kiểm tra từng ký tự trong xâu.

- `int CountNumberOfVowelsandConsonants()`: Đếm số lượng ký tự nguyên âm và phụ âm xuất hiện trong xâu hiện thời(st). Ví dụ chuỗi “java” có 2 nguyên âm “a” và 2 phụ âm “j” và “v”.

Gợi ý: Dùng switch case để đếm số lượng ký tự nguyên âm. Số phụ âm sẽ là độ dài của chuỗi trừ cho số lượng phụ âm.

Bài 3: Xây dựng lớp Xâu có thuộc tính riêng là : String st và các phương thức sau :

-`int ConvertStringNumbertToIntValue()`: Chuyển chuỗi số nguyên sang int value. Ví dụ “5646” thành int = 5646.

Gợi ý: Dùng Integer wrapper class để chuyển đổi kiểu string sang kiểu int.

`public void ReplaceCharacter(char ch1, char ch2)`: thay thế các ký tự ch1 bằng ch2. Ví dụ “shareprogramming.net” chuyển ‘e’ sang ‘*’ kết quả “shar*programming.n*t”.

Gợi ý: Có thể dùng `String#replace()` hoặc `StringBuilder` để lập và thao tác.

-`String ReverStringByWord()`: Đảo ngược các ký tự của chuỗi cách nhau bởi dấu cách mà không dùng thư viện. Ví dụ “I am developer ” => “developer am I”. Các ký tự bên trong chỉ cách nhau đúng một dấu khoảng cách.

Gợi ý: Cần loại bỏ dấu cách ở đầu và cuối câu, thao tác cắt chuỗi theo dấu cách và dùng `StringBuilder` hoặc `StringBuffer` để nối chuỗi.

`boolean CheckStringPalindrome()`: Chuỗi palindrome là chuỗi sau khi đảo ngược và chuỗi ban đầu hoàn toàn giống nhau, ví dụ “aba” là một chuỗi “aba”

Gợi ý: Đảo ngược chuỗi và so sánh với chuỗi ban đầu hoặc lặp và so sánh từng cặp chữ một.

-`String removeCharacterDuplicate()`: Xóa các ký tự xuất hiện nhiều hơn một lần trong xâu và chỉ giữ lại ký tự đầu tiên, ví dụ bananas => bans

Gợi ý: Sử dụng `HashSet` hoặc `ArrayList` lưu các ký tự đã xuất hiện trong lúc duyệt. Nếu ký tự được duyệt chưa có tổng `HashSet` hoặc `ArrayList` thì cộng ký tự đó vào chuỗi kết quả và thêm ký tự đó vào `HashSet` hoặc `ArrayList`. Lưu ý sử dụng `StringBuilder` hoặc `StringBuffer` để thao tác cộng chuỗi.

-String findLongestPalindromeSubstring(): Tìm substring palindrome dài nhất trong một chuỗi cho trước. Biết rằng chuỗi palindrome là chuỗi sau khi đảo ngược và chuỗi ban đầu giống nhau.